Fundamental Algorithm Techniques
Problem Set #8
Problem 1 — SCC and reversal .
1. Algorithm to compute the reversal \text{rev}(G) in O(V+E)
Idea: reverse every directed edge.
Algorithm:
Create a new graph \text{rev}(G) with the same vertices.

For every edge (u \to v) \in E, add edge (v \to u) to \text{rev}(G).

Time complexity:
Each vertex and edge is processed once → O(V+E).
2. Prove that \text{scc}(G) is acyclic
Proof:
In the SCC graph, each strongly connected component is compressed into a single node.

If there were a cycle between SCCs, then all nodes in the cycle would be mutually reachable.

This contradicts the maximality of SCCs.

Conclusion:
The SCC graph is a Directed Acyclic Graph (DAG).
3. Prove that \text{scc}(\text{rev}(G)) = \text{scc}(G)
Proof:
Two vertices u and v are in the same SCC if and only if:

u \leadsto v and v \leadsto u in G.

Reversing all edges does not change mutual reachability.

Therefore, SCCs remain unchanged.

Conclusion:
\text{scc}(\text{rev}(G)) = \text{scc}(G)
4. Reachability between SCCs
Statement:
If there is a path from u to v in G, then there is a path from S(u) to S(v) in \text{scc}(G).
Proof:
Every path in G corresponds to a sequence of SCCs.

Compressing vertices into SCCs preserves reachability.

Thus, reachability is preserved at the SCC level.

Conclusion:
Reachability in G implies reachability in the SCC graph.

Problem 2 — Euler Tour

1. Condition for existence of Euler tour

Statement:

A strongly connected directed graph has an Euler tour iff

\text{in-degree}(v) = \text{out-degree}(v) \quad \forall v \in V

Explanation:

Each time we enter a vertex, we must leave it.

Equality of in-degree and out-degree ensures edges can be paired.

2. Algorithm to find Euler tour in O(E)

Algorithm (Hierholzer's Algorithm):

- Start at any vertex.
- Follow unused edges until you return to the start.
- If unused edges remain, start a new cycle from a vertex in the current tour.
- Merge cycles.

Time complexity:

Each edge is used exactly once → O(E).

Problem 3 — Topological Sort

Given graph:

\{ A \to B,\ A \to C,\ B \to C,\ B \to D,\ C \to E,\ D \to E,\ D \to F,\ G \to F,\ G \to E \}

Topological sort starting from A

One valid ordering:

A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G

Another valid topological ordering

G \rightarrow A \rightarrow B \rightarrow D \rightarrow F \rightarrow C \rightarrow

Explanation

A node appears before all nodes it points to.

Multiple correct answers are possible.

Any order respecting dependencies is valid.